Sahulatcart — SRS for Uzair (Stream P)

Version 1.0 · 12 October 2026 · Source plan: PLAN.md · The other developer's file: SRS-Fazal.md

This is your copy of the build spec. Parts 0 and 1 are identical in both files: the rules both developers follow, the team split, and the eight interface contracts where your work meets Fazal's. Your stream's features follow, then the shared schedule, your rows of the acceptance matrix, and the appendices you need. Read Parts 0 and 1 first.

Table of contents

- Part 0 — Rules both developers follow
- Part 1 — Team split, dependency map, interfaces between the two streams
- Part 3 — Stream P, "Portal and operations": P1–P7 (yours)
- Part 4 — Week-by-week schedule and sync points
- Part 5 — Acceptance tests for Stream P
- Appendix B — New environment variables
- Appendix C — New migrations, in order

Stream M's features (M1–M7) and the Meta API appendix are in Fazal's file. You use them through the interfaces in Part 1.4 only.

Part 0 — Rules both developers follow

0.1 What already exists (do not rebuild)

The repo is an npm workspace: `shared/` (enums and types), `backend/` (Fastify + TypeScript), `admin/` (Next.js App Router), `db/migrations/` (plain SQL, applied in filename order), `site/` (static marketing site). Read `CLAUDE.md` and `docs/PITFALLS.md` before the first change. Working today: the Roman-Urdu sales bot, negotiation engine, COD and bank-transfer orders, order slips, the merchant portal (dashboard, inbox, orders, catalog, settings, notifications, analytics), durable webhook handling, retention, backups. Live: 1 merchant, 5 products, 0 orders.

0.2 Conventions that are not optional

Rule	Where it is enforced	Why
Money is **paisa** (integer) in the backend and DB; rupees only at the reply/UI edge (`rupees()` in orchestrator, `rs()` in admin).	Everywhere	One mixed unit corrupts every total.
`merchant_id` always comes from verified auth (`resolveCtx()` in `backend/src/lib/auth.ts`), never from the request body or query.	Every admin route	Tenant isolation.
By-id routes ownership-check and return **404**, not 403, when the row belongs to another merchant.	Every by-id route	Do not confirm another tenant's ids exist.
New SQL functions end with `revoke execute on function … from public, anon, authenticated;` .	Every migration	Postgres grants EXECUTE to PUBLIC by default; `/rest/v1/rpc` would expose it.

Rule	Where it is enforced	Why
Every `ReplySpec` kind has a Roman-Urdu `fallbackText()`.	`backend/src/llm/types.ts`, orchestrator	It ships when the LLM is down or the price guard trips.
Every LLM-composed reply passes `priceGuardOk()`. New reply kinds that always say the same thing go in `TEMPLATE_ONLY_KINDS` (no LLM call).	`backend/src/orchestrator/guard.ts`	Invariant 1: the LLM never decides prices.
Admin UI: plain CSS tokens from `admin/app/globals.css`; shared pieces from `admin/components/AppShell.tsx` (`PageHead`, `BackLink`, `EmptyState`, `StatusPill`, `humanize()`); lucide icons with `aria-hidden`; every input has a `<label>`; icon-only buttons have `aria-label`; money cells `td.num`; ids `.mono`. New colours need a token in **both** light and dark blocks.	Every admin page	Consistency and dark mode.
Admin fetches go through `admin/lib/api.ts` (`api()`, `apiJson()`), never straight to the backend. Show failures with `toast(await apiError(r), 'error')`; never toast "Saved" without checking `r.ok`.	Every admin page	The portal proxies same-origin through `admin/app/api/[...path]/route.ts`.
No secrets in chat, logs, URLs, or the browser. `pino` redaction is configured in `backend/src/lib/logger.ts`; add new secret field names there.	Backend	PITFALLS §1.
Tests run with `npm run test -w @app/backend` (never bare `npx vitest`). Typecheck: `npm run typecheck` and `cd admin && npm run typecheck`. Check the **exit code**, not the output.	Before every PR	PITFALLS §5.
`CHANGELOG.md` gets an entry **with every PR**, recording what was tried and failed, not only the result.	Every PR	Project rule in CLAUDE.md.

0.3 Branches, PRs, deploys

- Branch names: `feat/<stream>-<id>-<slug>`, e.g. `feat/m2-embedded-signup`, `feat/p1-signup`.
- One PR per feature id (M1, P3…). Small PRs inside a feature are welcome; name them `feat/p3-push-part1`.
- A PR needs: tests green, both typechecks green, a CHANGELOG entry, and the other developer's review. Merge to `main` deploys automatically: backend on `backend/**` or `shared/**` changes, admin on `admin/**` or `shared/**`. A docs-only PR deploys nothing.
- After a backend merge, confirm the deploy with `/healthz` (uptime resets) and `/readyz` (`db: ok`). `railway deployment list --service @app/backend` shows SUCCESS or FAILED; on FAILED read the build log before changing anything.

- Migrations: `npm run db:migrate` against the live database is run by **Fazal only**, after the PR that adds the migration is merged, and the CHANGELOG records the date. Never run a migration from a branch. Number them in order from `0009`; the sequence is fixed in Appendix C so the two streams do not collide.

0.4 Definition of done (every feature)

1. The "Done when" test of the feature passes against the **deployed** backend, not only locally.
2. Unit or flow tests exist for the new logic, in the file next to the source (`*.test.ts`).
3. The feature works in dark mode and at phone width (if it has UI).
4. CHANGELOG entry written; CLAUDE.md updated if the feature adds a rule the next person must know.
5. No new environment variable without an entry in `.env.example` and Appendix B.

0.5 Local setup for both

```
cp .env.example .env     # ask Fazal for the Supabase and Meta values; never paste them in chat
npm install
npm run build:shared
npm run dev:backend      # :8080
npm run dev:admin        # :3000
```

The replay sweeper and retention job only run with `NODE_ENV=production`, so a local backend never answers real customers even though it shares the live database. **Do not** set `NODE_ENV=production` locally. For Meta work, each developer uses the Meta app's test number and their own phone added as a test recipient (Meta allows 5).

Part 1 — Team split

1.1 Why two streams

The work splits cleanly along one line: everything that talks to **Meta** (tokens, Embedded Signup, templates, webhooks, quality, the message meter, the Meta dashboard and App Review) versus everything that is the **portal and shop operations** (signup, onboarding, alerts, order stages, opt-out, reliability, billing). The two streams share only a few interfaces (1.4), so they run in parallel and the calendar is about **9 weeks** instead of 14.

1.2 Streams and owners

Stream	Owner	Features	Weeks
M — Meta and messaging	**Fazal**	M1 per-merchant credentials and client · M2 Connect WhatsApp (Embedded Signup) · M3 connection page, quality webhooks, reconnect/disconnect · M4 templates, the window rule, fewer messages per sale · M5 message meter and cost controls · M6 platform owner console · M7 Meta dashboard and App Review	1–8
P — Portal and operations	**Uzair**	P1 self-service signup, password reset, automatic shop · P2 onboarding with readiness gates, delivery zones, COD toggle, greeting, attestations · P3 merchant alerts (Web Push + email) · P4 order stages and unpaid-order expiry · P5 opt-out enforcement and small gaps · P6 reliability: Sentry, rate limits, tests, CI · P7 billing	1–8

Fazal also owns Track A of the plan (Meta paperwork) because only the account holder can act there; M7 is the engineering half of it.

1.3 Dependency map

```
P1 signup ──► P2 onboarding ──► (needs M3 status endpoint for step 2) ──► P7 billing
                    │
M1 credentials ──► M2 connect ──► M3 status page ──► M4 templates ──► M5 meter ──► M6 console
                                                          │
P3 alerts (independent) ◄── M3 and M4 call the notify() helper from P3
P4 order stages ◄── uses sendTemplate() from M4 for the paid "order sent" button
P5 opt-out (independent)        P6 reliability (independent, but M-stream routes must adopt rate limits)
```

Nothing blocks week 1 for either developer. The first cross-dependency is in week 3 (P2 step 2 reads M3's status), and it is a read of one JSON field, so a stub is enough until M3 lands.

1.4 Interfaces between the streams (contracts; change only by agreement)

#	Provided by	Used by	Contract
IC-1	M1 (Fazal)	everything that sends	`sendText(phoneNumberId, to, body)`, `sendImage(...)`, `sendDocument(...)` in `backend/src/whatsapp/client.ts` keep their signatures. Internally they resolve the merchant's token from `phoneNumberId`. Callers never pass a token.
IC-2	M4 (Fazal)	P4 (Uzair)	`sendTemplate(phoneNumberId, to, templateName, params: string[])` returns `SendResult`. `sendButtons(phoneNumberId, to, body, buttons: {id, title}[])` for interactive replies.
IC-3	P3 (Uzair)	M3, M4, M5 (Fazal)	`notify(db, merchantId, { type, title, body, data })` in `backend/src/lib/notify.ts` writes the portal row **and** fans out to push and email per the merchant's preferences. All new notifications go through it.

#	Provided by	Used by	Contract
IC-4	M3 (Fazal)	P2 (Uzair)	`GET /api/v1/admin/whatsapp` returns `{ status: 'none'|'connecting'|'connected'|'needs_attention'|'disconnected', displayPhone, displayName, qualityRating, messagingTier, templates: [{name, status}], lastTestAt }`. Until M3 lands it returns `{ status: 'none' }`.
IC-5	P1 (Uzair)	M2 (Fazal)	After P1, every signed-in portal user has exactly one `merchants` row and one owner `merchant_users` row; `resolveCtx()` returns `merchantId`. M2 relies on nothing else.
IC-6	M5 (Fazal)	P7 (Uzair)	Table `message_usage(merchant_id, phone_number_id, period_month, outbound_count, inbound_count)` is written by the client on every send and read by billing.
IC-7	M6 (Fazal)	P7 (Uzair)	`isPlatformAdmin` in `MerchantCtx` becomes true for members of `platform_admins`; P7's "mark invoice paid" route checks it.
IC-8	both	both	`backend/src/orchestrator/orchestrator.ts` is touched by M4 (message merging, buttons) and P5 (opt-out guard). Keep each change in its own small PR and merge M4's first; P5's guard sits at the top of `runOrchestrator()` and does not touch state handlers.

Part 3 — Stream P: Portal and operations (Uzair)

P1 — Self-service signup, email confirmation, password reset, automatic shop

Purpose. Today a merchant can only be created by `POST /api/v1/admin/merchants` behind the shared admin token (`routes/admin.ts:57–67`), and a signed-in user with no `merchant_users` row is sent in a login loop (`lib/auth.ts:72`, `admin/lib/api.ts:10–12`). Merchants must be able to join on their own.

User story. As a shop owner, I sign up with my email and a password, confirm my email, log in, and find my own empty shop waiting. If I forget my password I can reset it by email.

Functional requirements

- FR-P1.1 Page `/signup` (new `admin/app/signup/page.tsx`): fields email, password (min 8), shop name, owner name, WhatsApp-capable personal phone (E.164, Pakistani `+92…` validated), two checkboxes (P2's attestations, FR-P2.9) and the terms/privacy links. Submits via Supabase `auth.signUp({ email, password, options: { emailRedirectTo: <portal>/auth/confirmed, data: { shopName, ownerName, phone } } })`. Supabase's "Confirm email" setting must be on (Fazal flips it in the dashboard; record in CHANGELOG).
- FR-P1.2 Gate: until `SIGNUP_OPEN=true`, the backend's `GET /api/v1/config` returns `signupOpen:false` and `signupAllowlist` is **not** exposed; the signup page still renders, but `POST /api/v1/signup/bootstrap` (FR-P1.4) refuses emails not in `SIGNUP_ALLOWLIST` (comma-separated, case-insensitive) with 403 `signup_closed`, and the page shows "Sahulatcart is in private pilot; email us to join". The Supabase user is created regardless (harmless), so the check is also applied in the page before calling `signUp` by asking `POST /api/v1/signup/check {email}` → `{allowed}`.
- FR-P1.3 Page `/auth/confirmed` handles the email link (Supabase puts the session in the URL hash; `supabase.auth.getSession()` after load), then calls bootstrap and redirects to `/onboarding`.
- FR-P1.4 `POST /api/v1/signup/bootstrap` (JWT required, **no** merchant membership required: a new auth path in `lib/auth.ts`, `resolveAuthUser()` that verifies the JWT only). In one transaction (`create_merchant_for_user` RPC, migration 0012): if a `merchant_users` row exists for this `auth_user_id` → return it (idempotent); else insert `merchants` (`business_name`, `owner_name`, `email`, `phone`, `status='active'`, `settings.botEnabled=false`, `settings.onboardingCompletedAt=null`) and `merchant_users` (`role='owner'`, `email`, `name`). Returns `{ merchantId, role }`. The RPC ends with the revoke line. The default `settings` in migration 0001 has `botEnabled:true`; the RPC sets it false explicitly so a new shop never answers before onboarding (plan decision; today's default is the trap the old plan noted).
- FR-P1.5 The existing `POST /api/v1/admin/merchants` stays for the platform admin but also sets `botEnabled=false`.
- FR-P1.6 Password reset: `/login` gets "Forgot password?" → `/reset` page → `auth.resetPasswordForEmail(email, { redirectTo: <portal>/auth/reset })` → `/auth/reset` page reads the recovery session and calls `auth.updateUser({ password })`.
- FR-P1.7 `api()` in `admin/lib/api.ts`: on 401 with body `{ error: 'no_membership' }` (new, returned by `resolveCtx` callers when the JWT is valid but no membership exists) redirect to `/auth/confirmed` (which bootstraps) instead of `/login`, breaking the loop.
- FR-P1.8 Rate limits on `/signup/*` and the Supabase calls are P6's; the page disables the button while a request is in flight.
- FR-P1.9 The marketing site's "Login" links stay; add a "Start free" button pointing at `<portal>/signup` on the home page hero and pricing page (site is static HTML; edit `site/index.html` and `site/pricing.html`; `site/assets/site.js` rewrites railway links to localhost for local work).

Data (migration 0012). `create_merchant_for_user(p_auth_user uuid, p_email text, p_shop text, p_owner text, p_phone text) returns table(merchant_id uuid, role user_role)` as described; also `alter table merchants alter column settings set default` with `botEnabled:false` so future direct inserts are safe.

API. `POST /api/v1/signup/check {email} → {allowed}` ; `POST /api/v1/signup/bootstrap → {merchantId, role}` ; `GET /api/v1/config` adds `signupOpen` .

UI. `/signup` , `/reset` , `/auth/confirmed` , `/auth/reset` ; `/login` adds the two links. All use the existing login page's form styles. Error copy is specific: "That email already has an account; log in or reset your password" (Supabase returns `user_already_exists`).

How it works (sequence). signup page → `check` → `signUp` → Supabase sends the confirm email → link → `/auth/confirmed` → session → `bootstrap` → RPC → `/onboarding` . Returning user: `/login` → session → `GET /api/v1/admin/me` → 200 → dashboard; if 401 `no_membership` → bootstrap → dashboard.

Security. The RPC is the only path that creates merchants from a JWT; it runs as service role and takes `auth_user_id` from the verified token, never from the body. Shop name and owner name are validated through `lib/validation.ts` (length 2–80, no control characters).

Tests. `auth.test.ts` (new): JWT without membership → `no_membership` ; bootstrap twice → one merchant; allow-list closed → 403; open → 200. Fake-DB twin of the RPC in `test-support/fake-db.ts` (as for 0006/0008). Manual: full signup on the deployed portal with a Gmail "+tag" address.

Done when. A brand-new email on the allow-list signs up, confirms, lands on onboarding, and has exactly one merchant and one owner row; a second login finds the same shop; a non-allow-listed email gets the pilot message; password reset works end to end.

P2 — Onboarding with readiness gates, delivery zones, COD toggle, greeting, attestations

Purpose. `admin/app/onboarding/page.tsx` has five steps and "Go live" only sets `botEnabled` / `onboardingCompletedAt` (`:27`). It must become seven checked steps, and the bot must be impossible to enable until every check passes on the server.

Functional requirements

- FR-P2.1 `GET /api/v1/admin/readiness` returns `{ steps: { profile, whatsapp, catalog, delivery, payment, persona, testOrder }, ready: boolean }` , each step `{ ok: boolean, reason?: string }` , computed server-side:
- `profile` : `business_name` , `phone` , `category` set; `business_hours` has at least one day.
- `whatsapp` : IC-4 status is `connected` .
- `catalog` : ≥1 active product with price > 0.
- `delivery` : ≥1 serviceable `delivery_zones` row **or** `settings.defaultDeliveryCharge` set explicitly (a new flag `settings.deliveryConfigured=true` written by the step).
- `payment` : `settings.codEnabled` true **or** ≥1 active `bank_accounts` row.
- `persona` : `bot_persona.name` and `.style` set.
- `testOrder` : ≥1 order with `status in ('confirmed','awaiting_payment','paid')` whose customer `wa_id` equals `merchants.phone` (the owner's own phone).
- FR-P2.2 `PATCH /api/v1/admin/settings` rejects `botEnabled: true` with 409 `not_ready` and the failing steps unless `ready` is true. The existing settings page's bot toggle shows the reason.
- FR-P2.3 The wizard (same page, rewritten) shows the seven steps as a checklist with live status from `readiness` ; each step is a panel: 1. **Shop**: name, owner name, phone, category (select from a fixed list: clothing, electronics, grocery, cosmetics, home, other), business hours (per-day open/close, "closed" toggle), logo upload (reuse the product image upload route pattern; new `POST /api/v1/admin/settings/logo` , bucket `product-images` , key `merchants/<id>/logo.<ext>`). Saved to `merchants` . 2. **WhatsApp**: embeds M2's connect component (import from `admin/app/settings/whatsapp/`); shows IC-4 status. 3. **Products**: the existing add-product form and import buttons, inline; list of products so far. 4. **Delivery**: a table of zones (area, city, charge, serviceable, ETA text) with add/edit/delete, plus the default charge for unlisted areas and a free-delivery threshold. New routes: `GET/POST /api/v1/admin/delivery-zones` , `PATCH/DELETE /api/v1/admin/delivery-zones/:id` (today `GET` is the only one, `routes/admin.ts:461`). Validation: `area_name` 2–60 chars, charge ≥ 0

paisa, no duplicate `area_name` per merchant (case-insensitive). 5. **Payment**: COD on/off (`settings.codEnabled` , **now enforced**: `handlePayment()` offers only the enabled methods; if neither is enabled the readiness step fails), bank accounts (existing add/delete), payment instructions text (`settings.paymentInstructions` , shown with bank details). 6. **Bot**: name, style (narm/standard/sakht), greeting text (`bot_persona.greeting` , **now used**: the `greeting` reply spec gets `greetingText` and the fallback uses it when set), negotiation defaults (existing fields). 7. **Test order**: instructions "Message your shop number from your own phone and buy something"; a **Send test message** button (M3's `/whatsapp/test`) to prove the number works; the step turns green when the test order exists. Then a **Go live** button that calls `PATCH settings {botEnabled:true}` ; on 409 it highlights the failing steps.

- FR-P2.4 The dashboard's "Finish setting up" card (`admin/app/page.tsx:18–40`) links to the first failing step and disappears exactly when `ready` and `botEnabled` are both true.
- FR-P2.5 Business hours are used: outside hours the bot still answers (buyers message at night), but the greeting fallback appends "Hum {open}–{close} khule hain" when hours exist. (Small, but the field was dead.)
- FR-P2.6 `GET /api/v1/admin/settings` returns the readiness block too, so the settings page can show why the bot toggle is disabled.
- FR-P2.7 Delivery zones are also editable later from `/settings` (same component).
- FR-P2.8 The wizard is resumable: every panel saves on its own; nothing is lost on refresh.
- FR-P2.9 Attestations at signup (P1 form, stored on bootstrap): `settings.attestations = { noProhibitedGoods: true, optInOnly: true, at: <iso> }` . The wizard's step 1 shows them as accepted with the date. Readiness fails if missing (covers merchants created by the admin route).

Data (migration 0013). `alter table merchants add column logo_url text;` and nothing else: `delivery_zones` , `bank_accounts` , `bot_persona` , `business_hours` , `settings` exist.

UI notes. Use `PageHead` , a left step list with `StatusPill` (done / to do / problem), the active panel on the right; on phone width the list collapses to a top stepper. Every input labelled.

Tests. `readiness.test.ts` over FakeDb: each step's pass/fail; `PATCH settings botEnabled` 409 until ready; delivery-zone CRUD with duplicate rejection; flow test: `codEnabled=false` offers bank only; greeting fallback uses the merchant's text; an order from `merchants.phone` flips `testOrder` .

Done when. A new tester reaches "live" only through all seven steps on the deployed portal; the dashboard card disappears at that moment; with COD off, the bot never offers COD.

P3 — Merchant alerts: Web Push and email (the `notify()` helper)

Purpose. Notifications are written to `notifications` with `channel='portal'` only (`orchestrator.ts:529` , `order-service.ts:240` , `payment-service.ts:82`); a merchant learns of an order only while the portal is open. Push and email are free; WhatsApp alerts cost money (M4 rule).

Functional requirements

- FR-P3.1 `backend/src/lib/notify.ts` exports `notify(db, merchantId, { type, title, body, data, userId? })` (IC-3). It writes the portal row (as today), then reads the merchant's owner/manager members' `notification_prefs` (`{ push: true, email: true, whatsapp: false }` default) and fans out. All existing call sites are changed to call it (the three above plus `template_status` , `system` , `bot_needs_help` producers in M-stream).
- FR-P3.2 **Web Push**: `admin/public/sw.js` service worker; `admin/public/manifest.json` (`display: 'standalone'` , icons from `site/assets/logo-1024.png` resized to 192/512, theme colour teal); `<link rel="manifest">` and iOS meta tags in `admin/app/layout.tsx` . A "Turn on phone notifications" button on the dashboard and in Settings → Notifications requests permission and posts the `PushSubscription` (`endpoint` , `keys.p256dh` , `keys.auth`) to `POST /api/v1/admin/push-subscriptions` ; `DELETE` on unsubscribe. iPhone: show the instruction "Add to Home Screen first" when `navigator.standalone` is false and the UA is iOS.

- FR-P3.3 Backend sends pushes with the `web-push` package (VAPID keys in env, Appendix B). Payload `{ title, body, url }`; the service worker shows the notification and opens `url` on click (order page, inbox thread, or WhatsApp settings). A 404/410 from the push service deletes the subscription.
- FR-P3.4 **Email** through Resend's HTTP API with plain `fetch` (no SDK; key in env). From `alerts@sahulatcart.com` (Cloudflare Email Routing handles inbound; Resend needs the domain verified with DKIM/SPF records added in Cloudflare — Fazal adds them; until then send from Resend's test sender). One email per notification, plain text + a minimal HTML version, with the portal link. Batch: if more than 5 notifications in 5 minutes for one merchant, send one digest email instead.
- FR-P3.5 **WhatsApp** channel: off by default; when a merchant turns it on in Settings → Notifications, `notify()` sends a free-form text to `merchants.phone` from the shop's own number, with the note that it is counted against the free 1,000 (M5). Never a template.
- FR-P3.6 Settings → Notifications page: per-channel toggles, the push button with its state, the email address used (owner's), and a "Send me a test" button per channel (`POST /api/v1/admin/notifications/test {channel}`).
- FR-P3.7 Failures never block the caller: `notify()` awaits the portal insert and fires push/email with `void … catch(log)` .

Data (migration 0014).

```
create table push_subscriptions (
  id uuid primary key default gen_random_uuid(),
  merchant_id uuid not null references merchants(id) on delete cascade,
  member_id uuid not null references merchant_users(id) on delete cascade,
  endpoint text not null unique, p256dh text not null, auth text not null,
  user_agent text, created_at timestamptz not null default now(), last_used_at timestamptz
);
```

Tests. `notify.test.ts` : fan-out per prefs; push 410 removes the subscription; digest after 5 in 5 minutes; email failure does not fail the caller. Manual: Android Chrome and iOS home-screen push on the deployed portal.

Done when. A tester places an order with the portal closed and gets a phone notification within a minute on Android and on an iPhone with the portal on the home screen; the email arrives too.

P4 — Order stages and unpaid-order expiry

Purpose. Orders stop at paid / COD collected; `preparing` , `dispatched` , `delivered` exist in the enum but nothing sets them. Bank orders with no screenshot stay `awaiting_payment` forever, holding stock.

Functional requirements

- FR-P4.1 Transitions, each a single conditional UPDATE (the pattern in `payment-service.ts`) plus an `order_status_history` row: `confirmed|paid → preparing → dispatched → delivered` ; `cancel` allowed from any non-terminal state and calls `release_stock()` . Routes: `POST /api/v1/admin/orders/:id/preparing` , `/dispatched { courier?, trackingRef?, etaText? }` , `/delivered` . 409 on an invalid transition. Staff may set stages; only managers may cancel (per spec 09 §2.3).
- FR-P4.2 Dispatch stores `courier` , `tracking_ref` , `eta_text` on the order (new columns) and sets `pending_buyer_notice='order_sent'` when the buyer's window is closed (normal case), or sends the free-form line when it is open. The order page then shows M4's paid button for the notice.
- FR-P4.3 **Unpaid expiry job** `expireUnpaidOrders()` in `backend/src/lib/jobs.ts` , run hourly like `startRetentionJob()` (production only, `index.ts`): orders with `status='awaiting_payment'` , `payment_status='unpaid'` , no `payment_claims` row, `placed_at < now() - interval '24 hours'` → status `cancelled` , `cancelled_reason='payment not received in 24h'` , `release_stock()` , history row, `notify()` to the merchant, and a free-form text to the buyer **only if their window is open** (usually not; then nothing, per the cost rule). The job is idempotent (the status filter).

- FR-P4.4 `settings.unpaidExpiryHours` (default 24, 6–72) editable in Settings → Payment.
- FR-P4.5 Orders list gains stage filters and a stage column; the order page shows the stage stepper with the next-action button; `humanize()` covers the new statuses.
- FR-P4.6 Dashboard "to do" counts: awaiting screenshot review, to prepare, to dispatch.

Data (migration 0015). `alter table orders add column courier text, add column tracking_ref text, add column eta_text text;`

Tests. Transition matrix test (every from/to pair → 200 or 409); expiry job over FakeDb (stock returns, notification written, a claimed order is untouched, a run twice changes nothing); dispatch with window open sends one free-form message, with window closed sets the pending notice.

Done when. On the deployed backend a bank order with no screenshot is cancelled after 24 hours with its stock back; a tester moves an order confirmed → preparing → dispatched → delivered from the portal and the history shows four rows.

P5 — Opt-out enforcement, STOP keyword, and small gaps

Functional requirements

- FR-P5.1 At the top of `runOrchestrator()` (IC-8): if `customers.wa_opt_in_status='out'`, store the message, do not reply, do not classify (no LLM call), and bump `unread_count` so the merchant sees it in the inbox with an "opted out" badge. Exception: the exact keywords `start` / `shuru` re-opt the customer in (`wa_opt_in_status='in'`, `wa_opt_in_source='keyword'`, `wa_opt_in_at`) and get one free-form welcome.
- FR-P5.2 `STOP_RE` beside `CANCEL_RE`: `^\s*(stop|unsubscribe|band karo|bandh karo|msg mat karo)\s*$` (whole message, multi-word or unambiguous per the convention), matched before any LLM call; sets opt-out, sends one goodbye (the existing `stop` behaviour at `orchestrator.ts:284–287`), and never replies again.
- FR-P5.3 Opt-in capture: on the first inbound message from a new customer set `wa_opt_in_status='in'`, `wa_opt_in_source='inbound_message'`, `wa_opt_in_at=now()` (today it stays `unknown`). The inbox customer panel shows the opt-in state and date.
- FR-P5.4 **Delete a product**: `DELETE /api/v1/admin/products/:id` sets `is_active=false` and `deleted_at` (new column) rather than deleting (order items reference it); the catalog hides deleted rows; `resolveProduct()` already filters on `is_active`.
- FR-P5.5 **Edit the bot name** after onboarding: Settings → Bot panel edits `bot_persona.name` and `.greeting` (the wizard's step 6 component reused).
- FR-P5.6 **Dashboard "today" in Pakistan time**: `routes/admin.ts:18–21` computes the day in UTC; use `Asia/Karachi` (compute the start of day with `Intl.DateTimeFormat` parts or a 5-hour offset helper in `lib/time.ts`, tested at 23:30 and 00:30 PKT).
- FR-P5.7 **Stock double-order guard**: a buyer who already has an open order for a tracked product cannot start a second order for more than the remaining stock (the first order's reservation already reduced `products.stock`, so the existing "sirf N available" check holds; add a flow test that proves it and fix if it fails).
- FR-P5.8 **Customer block**: `customers.is_blocked` is honoured like opt-out (store, no reply), with a Block/Unblock button in the inbox customer panel (`POST /api/v1/admin/customers/:id/block`).

Data (migration 0016). `alter table products add column deleted_at timestamptz;`

Tests. Flow: opted-out customer gets no reply and no LLM call (assert the fake LLM was not called); `STOP` works in every state; `start` re-opts; blocked customer silent; first message sets opt-in; today-in-PKT boundary tests.

Done when. The flow tests pass and the inbox shows opt-in/blocked states for the pilot merchant's customers.

P6 — Reliability: error alerting, rate limits, lockout fix, tests and CI

Functional requirements

- FR-P6.1 **Sentry**: `@sentry/node` in the backend (`index.ts`, DSN from env, `environment=NODE_ENV`, release = git SHA from Railway's `RAILWAY_GIT_COMMIT_SHA`), Fastify error hook reports 5xx and unhandled rejections; `@sentry/nextjs` in the admin for client and route-handler errors. Redact request bodies (they can hold addresses and phone numbers): `sendDefaultPii: false`, and a `beforeSend` that drops `request.data`. Alert rule in Sentry: email the owner on any new issue.
- FR-P6.2 **Uptime**: an external check (Better Stack or UptimeRobot free tier, configured by Fazal) on `GET /readyz` every 5 minutes, **never** `/readyz?llm=1`.
- FR-P6.3 **Rate limits** with `@fastify/rate-limit`, keyed by merchant id when authenticated else by IP: `/api/v1/signup/*` 10/min/IP; `/api/v1/admin/login` 10/min/IP; `/api/v1/admin/*` 300/min per merchant; `/api/v1/admin/whatsapp/connect` 5/min per merchant; the webhook route is exempt (Meta bursts). 429 bodies follow the existing `{ error }` shape.
- FR-P6.4 **Admin-token lockout fix** (`lib/auth.ts:40–54`): the counter becomes per source IP (`X-Forwarded-For` first hop, Railway sets it) instead of global, so a stranger's ten guesses cannot lock the owner out; still constant-time compare.
- FR-P6.5 **Tests that are missing**: `routes/admin.test.ts` booting Fastify with FakeDb via `app.inject()`: auth (JWT with/without membership, admin token, bad token), **cross-tenant** (merchant A requests merchant B's order/product/conversation by id → 404; list routes never include B's rows), manager-only routes refuse staff; `webhook.route.test.ts` posting a signed real payload through the HTTP route (signature → claim → process → reply sent). Add `.or()` / `.range()` to `fake-db.ts` if a route needs them.
- FR-P6.6 **CI**: `.github/workflows/ci.yml` on every PR: `npm ci`, `npm run build:shared`, `npm run typecheck`, `cd admin && npm run typecheck`, `npm run test -w @app/backend`, and `next lint` for the admin (add `eslint-config-next`; fix the existing warnings once). Branch protection on `main` requires the workflow (Fazal sets it in GitHub).
- FR-P6.7 **Single instance stays**; add a startup assertion that `RAILWAY_REPLICA_ID` (if present) is the only replica, logging a loud warning otherwise, and a CLAUDE.md note on the DB-lock change needed for a second replica.

Tests. The tests *are* the feature; CI must be green on the PR that adds them.

Done when. A deliberate `throw` behind a test route in staging reaches the owner's inbox from Sentry within five minutes; the cross-tenant test is in the suite; CI blocks a PR with a failing test.

P7 — Billing: plans, trial, limits, usage page, invoices

Purpose. `merchants.plan` (`pilot|basic|pro`) exists and is never read. Launch needs a free pilot, then a paid plan, enforced softly (never a silent cut-off), paid by bank transfer and marked by hand in the platform console (M6).

Functional requirements

- FR-P7.1 Plan definitions in `shared/src/plans.ts`: `pilot` (30 days free, 300 conversations/month), `basic` (price in paisa from config `PLAN_BASIC_PRICE_PAISA`, 1,000 conversations/month), `pro` (`PLAN_PRO_PRICE_PAISA`, 5,000). A "conversation" is a `conversations` row with ≥1 inbound message in the period month. Prices are config so the pilot can set them without a deploy.
- FR-P7.2 `subscriptions` table (migration 0017): `merchant_id`, `plan`, `status` (`trial|active|past_due|expired`), `trial_ends_at`, `current_period_start/end`, `created_at`. Bootstrap (P1) creates a `trial` row with `trial_ends_at = now()+30 days`.
- FR-P7.3 `invoices` table: `merchant_id`, `subscription_id`, `period_start/end`, `amount_paisa`, `status` (`due|paid|void`), `reference` (merchant's transfer reference), `paid_at`, `marked_by_platform_admin`. A monthly job (`lib/jobs.ts`, beside P4's) creates the next invoice 7 days before `current_period_end` and notifies the merchant with the bank details of **Nubrix** (config `PLATFORM_BANK_DETAILS`, multi-line).
- FR-P7.4 Merchant page `/billing`: plan, trial/period dates, usage this month (conversations from the plan limit; messages from M5's meter), invoices with status and the reference field ("I've paid, reference …" → `POST /api/v1/admin/invoices/:id/claim {reference}` → status stays `due` with the reference shown to the platform

admin), and plan change requests (`POST /api/v1/admin/subscription/ request {plan}` → notification to the platform admin; no self-service upgrade yet).

- FR-P7.5 Platform console (M6) gains an Invoices tab: due invoices with claimed references, **Mark paid** (`POST /api/v1/platform/invoices/:id/paid` , IC-7) which extends `current_period_end` by one month and sets `active` ; **Change plan**.
- FR-P7.6 Enforcement: `expired` or `past_due` (invoice unpaid 7 days after `current_period_end`) → the bot switches to **handoff-only** (M5's `afterFreeQuota='handoff'` path reused via a shared `botMode(merchant)` helper returning `'sell'|'handoff'|'off'`), the dashboard shows a banner with the invoice, nothing is deleted, inbound messages are still stored and the merchant can still reply by hand. Conversation limit reached → the same handoff-only mode until the period resets.
- FR-P7.7 Trial end: 7 days before, a notification and email; at the end, status `past_due` with the first invoice `due` .

Data (migration 0017). `subscriptions` , `invoices` , indexes on `merchant_id` ; a `plan_limits` check is code, not SQL.

Tests. Trial creation at bootstrap; invoice creation job; mark-paid extends the period; enforcement flips `botMode` ; a merchant in handoff-only mode still receives and stores messages; the flow test proves a buyer mid-checkout completes before the mode applies.

Done when. A shop on an expired plan still receives messages, the merchant sees why the bot stopped, and marking the invoice paid in the console restores selling within one message.

Part 4 — Week-by-week schedule and sync points

Week	Fazal (M)	Uzair (P)	Sync
1	M1 crypto + credentials + client refactor; M7 steps 1–4 as soon as Track A1–A3 allow	P1 signup, confirm, reset, bootstrap RPC	Mon: agree IC-1..IC-8 line by line. Fri: both PRs reviewed.
2	M2 connect route + page (mocked Meta first, then the test WABA)	P2 readiness endpoint, delivery-zone CRUD, wizard shell	IC-4 stub agreed (`{status:'none'}`).
3	M3 status page, account/quality webhooks, disconnect	P2 wizard panels 1–7, COD enforcement, greeting	P2 step 2 embeds M2's component: pair for an hour.
4	M4 templates, window rule, message merging, buttons; **record videos, submit App Review** (M7 step 6)	P3 `notify()` , push, email, settings page	M3/M4 switch their notifications to `notify()` (IC-3).
5	M5 meter, usage endpoint, dashboard card, after-quota mode	P4 order stages, dispatch notice, unpaid expiry job	P4's dispatch uses `sendTemplate` (IC-2).
6	M6 platform console; suspension enforcement	P5 opt-out, STOP, small gaps; P6 Sentry, rate limits, lockout fix	IC-8: M4's orchestrator PR merged before P5's.
7	Pilot shops onboarded as testers; M7 approval-day prep; fix-up	P6 tests + CI; P7 schema, trial, invoices	IC-6/IC-7: billing reads the meter; console gets Invoices tab.
8	Pilot support; Meta review follow-ups	P7 billing page, enforcement, jobs	Pilot stop rules reviewed daily (PLAN.md B6).
9–10	Pilot (both). Zero wrong prices, zero duplicate orders, zero cross-shop leaks, or onboarding pauses.		Weekly: measure messages per sale and cost per shop.

Daily: a 10-minute call. Every PR reviewed by the other developer within one working day. The CHANGELOG is the shared log; disagreements on an interface are settled by editing Part 1.4 in a PR.

Part 5 — Acceptance tests for Stream P

#	Scenario	Expected	Feature
T1	New allow-listed email signs up, confirms, logs in	One merchant, one owner, lands on onboarding, bot off	P1
T2	Same user logs in again	Same shop, no second merchant	P1
T3	Email not on the allow-list	Pilot message, no shop	P1
T16	Go live with COD off and no bank account	409, step 5 highlighted	P2
T17	All seven steps done	Go live succeeds, dashboard card gone	P2
T18	Order placed with portal closed	Push on Android and iOS (home screen) and an email within a minute	P3
T19	Bank order, no screenshot, 25 h	Cancelled, stock back, merchant notified, buyer untouched	P4
T20	confirmed → preparing → dispatched → delivered	Four history rows; dispatch with window closed sets the paid notice	P4
T21	Buyer sends "STOP", then "hi"	Goodbye once, then silence; inbox shows opted out	P5
T22	Merchant A requests merchant B's order id	404; list routes show none of B's rows	P6
T23	Ten bad admin-token guesses from IP X	Owner from IP Y still works	P6
T24	Throw in a test route	Sentry email within five minutes	P6
T25	Trial ends, invoice unpaid 7 days	Bot handoff-only, messages still stored; mark paid restores selling	P7

Appendix B — New environment variables

Variable	Service	Set by	Purpose
`TOKEN_ENCRYPTION_KEY`	backend	Fazal (exists, unused)	32-byte base64 key for M1; required in production
`META_APP_ID`	backend, admin (public via `/config`)	Fazal	Embedded Signup and token exchange
`META_EMBEDDED_SIGNUP_CONFIG_ID`	backend (public via `/config`)	Fazal	Embedded Signup configuration
`META_GRAPH_API_VERSION`	backend	Fazal	made required (no default) in M7
`META_UTILITY_RATE_PAISA`	backend	Fazal	shown wherever a paid message is offered; default 420
`SIGNUP_OPEN` / `SIGNUP_ALLOWLIST`	backend	Fazal	P1 gate
`VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY` / `VAPID_SUBJECT`	backend (public key via `/config`)	Uzair generates, Fazal sets	Web Push
`RESEND_API_KEY` / `ALERT_EMAIL_FROM`	backend	Fazal	email alerts
`SENTRY_DSN` (backend), `NEXT_PUBLIC_SENTRY_DSN` (admin)	both	Fazal	error alerting
`PLAN_BASIC_PRICE_PAISA` / `PLAN_PRO_PRICE_PAISA` / `PLATFORM_BANK_DETAILS`	backend	owner	billing
removed: `META_SYSTEM_USER_TOKEN`	backend	Fazal, after the pilot merchant reconnects	per-merchant tokens only

Every variable is added to `.env.example` with a comment, and declared as `preserve()` in `.railway/railway.ts` where the services need it, in the same PR that reads it.

Appendix C — New migrations, in order

File	Stream	Contents
`0009_whatsapp_connect.sql`	M	`whatsapp_secret_kind + 'pin'`; `whatsapp_connect_attempts`; `whatsapp_numbers.last_test_at`, `.attention_message`; index on `whatsapp_secrets(merchant_id, kind)`
`0010_message_templates.sql`	M	`message_templates`; `orders.pending_buyer_notice`
`0011_message_usage.sql`	M	`message_usage`; `bump_message_usage()` with revoke
`0012_self_signup.sql`	P	`create_merchant_for_user()` with revoke; `merchants.settings` default `botEnabled:false`
`0013_onboarding.sql`	P	`merchants.logo_url`
`0014_push_subscriptions.sql`	P	`push_subscriptions`
`0015_order_stages.sql`	P	`orders.courier`, `.tracking_ref`, `.eta_text`
`0016_products_soft_delete.sql`	P	`products.deleted_at`
`0017_billing.sql`	P	`subscriptions`, `invoices`

Each migration is applied to the live database by Fazal after its PR merges; `fake-db.ts` gets a JS twin of every new RPC in the same PR.